



CO₂ Emission Analysis Using Machine Learning

Prepared by: Group A

Tasneem Ashour – 12323555

Berlanti Masalha – 12323064

Batool Basalat – 12323591

Supervisor: Dr. Adnan Salman

An-Najah National University

Faculty of Information Technology and Artificial Intelligence

Department of Computer Science

May 20, 2026

Table of Contents

List of Figures..... 3

List of Tables 4

Introduction..... 5

1. Dataset Description 6

 1.1. Dataset Overview 6

 1.2. Dataset Features..... 6

2. Exploratory Data Analysis (EDA) 7

 2.1. Distribution of CO2 Emissions..... 7

 2.2. Relationship Between Vehicle Specifications and CO2 Emissions 9

 2.2.1. engine size 9

 2.2.2. cylinders..... 10

 2.3. Consumption Analysis..... 11

 2.3.1. City Fuel Consumption 11

 2.3.2. Highway Fuel Consumption..... 12

 2.3.3. Combined Fuel Consumption..... 13

 2.4. Fuel Type Analysis 14

 2.5. Correlation Analysis..... 15

 2.6. Average CO2 Emissions by Vehicle Class..... 16

3. Data Preprocessing..... 17

 3.1. Handling Missing Values..... 17

 3.2. Removing Duplicate Records..... 18

 3.3. Feature Encoding..... 18

 3.4. Train-Test Split 19

 3.5. Feature Scaling..... 20

4. Model Training..... 22

 4.1. Linear Regression..... 22

 4.2. Ridge Regression..... 23

 4.3. Support Vector Machine (SVR)..... 24

 4.4. Decision Tree Regressor 26

 4.5. Random Forest Regressor 28

5. Evaluation 30

 5.1. Linear Regression Evaluation 30

 5.2. Ridge Regression Evaluation 31

 5.3. Support Vector Regression (SVR) Evaluation..... 32

5.4.	Decision Tree Evaluation	33
5.5.	Random Forest Evaluation	34
5.6.	Overall Comparison	34
6.	Feature Importance and Coefficient Analysis.....	36
7.	Development Considerations.....	38
7.1.	Data Quality	38
7.2.	Categorical Data Processing	38
7.3.	Preventing Data Leakage	38
7.4.	Feature Scaling Requirements.....	38
7.5.	Hyperparameter Optimization	38
7.6.	Computational Performance	38
7.7.	Model Generalization	38
8.	Conclusion	39

List of Figures

Figure 1. Distribution of CO2..... 7

Figure 2. CO2 Emissions Boxplot..... 8

Figure 3. Engine Size vs CO2 Emission..... 9

Figure 4. Cylinders vs CO2 Emissions 10

Figure 5. City Fuel Consumption vs CO2 Emissions..... 11

Figure 6. Highway Fuel Consumption vs CO2 Emissions 12

Figure 7. Combined Fuel Consumption vs CO2 Emissions 13

Figure 8. Average CO2 Emissions by Fuel Type 14

Figure 9. Correlation Matrix of Numerical Features 15

Figure 10. Average CO2 Emissions by Vehicle Class 16

Figure 11. Actual vs Predicted and Residual Plots for the Linear Regression Model. 30

Figure 12. Actual vs Predicted and Residual Plots for the Ridge Regression Model. 31

Figure 13. Actual vs Predicted and Residual Plots for the SVR Model. 32

Figure 14. Actual vs Predicted and Residual Plots for the Decision Tree Model..... 33

Figure 15. Actual vs Predicted and Residual Plots for the Random Forest Model..... 34

Figure 16. Feature Importance and Coefficient Analysis for Machine Learning Models Predicting CO2 Emissions..... 36

List of Tables

Table 1. Vehicle Dataset Features, Descriptions, and Data Types 6

Table 2. SVR Hyperparameters and Tested Values 25

Table 3. Decision Tree Hyperparameters and Tested Values..... 26

Table 4. Random Forest Hyperparameters and Tested Values..... 28

Table 5. Comparison of Regression Model Performance Metrics 30

Introduction

Climate change and environmental pollution have become major global concerns in recent years. One of the primary contributors to environmental pollution is carbon dioxide (CO₂) emissions generated by vehicles. The amount of CO₂ emitted by a vehicle depends on several factors, including engine size, fuel type, fuel consumption, and vehicle class. Understanding the relationship between these factors and vehicle emissions can help governments, manufacturers, and individuals improve fuel efficiency and reduce environmental impact.

This project aims to analyze vehicle emission data and develop machine learning models capable of predicting CO₂ emissions based on different vehicle characteristics. Several data preprocessing techniques were applied before model training, including handling missing values, removing duplicate records, encoding categorical variables, feature scaling, and performing Exploratory Data Analysis (EDA).

After preprocessing, multiple regression models were trained and evaluated, including Linear Regression, Ridge Regression, Support Vector Regression (SVR), Decision Tree Regressor, and Random Forest Regressor. The models were compared using several evaluation metrics such as R² Score, Mean Absolute Error (MAE), and Root Mean Squared Error (RMSE) to determine the most effective predictive model.

The main objective of this project is to compare the performance of different machine learning algorithms and identify the factors that most strongly influence vehicle CO₂ emissions. The project demonstrates how data analysis and machine learning techniques can support environmental studies, fuel efficiency analysis, and intelligent decision-making in sustainable transportation systems.

1. Dataset Description

1.1. Dataset Overview

The dataset used in this project was collected from a Canadian government vehicle emissions database. It contains information about 7,385 vehicles described by 12 different features related to vehicle specifications and fuel consumption.

Each row in the dataset represents a single vehicle model, while the columns describe different characteristics such as engine size, number of cylinders, transmission type, fuel type, and fuel consumption measurements. The dataset contains both numerical and categorical variables, making it suitable for machine learning, statistical analysis, and predictive modeling tasks.

The main objective of using this dataset is to analyze the relationship between vehicle characteristics and carbon dioxide emissions. The target variable in this project is `co2_emissions`, which measures the amount of CO2 emitted by a vehicle in grams per kilometer (g/km). Since the target variable is continuous numerical data, the problem is treated as a supervised machine learning regression task.

1.2. Dataset Features

Feature	Description		type
<code>make</code>	Vehicle manufacturer such as Toyota, Ford, or BMW		Categorical
<code>model</code>	Specific model name of the vehicle		Categorical
<code>vehicle_class</code>	Vehicle category such as SUV, Sedan, or Compact		Categorical
<code>engine_size</code>	Engine size measured in liters		Numerical
<code>cylinders</code>	Number of engine cylinders		Numerical
<code>transmission</code>	Type of transmission system		Categorical
<code>fuel_type</code>	Type of fuel used by the vehicle		Categorical
	Fuel Code	Fuel Type	
	X	Regular Gasoline	
	Z	Premium Gasoline	
	D	Diesel	
	E	Ethanol (E85)	
<code>fuel_consumption_city</code>	Fuel consumption during city driving		Numerical
<code>fuel_consumption_hwy</code>	Fuel consumption during highway driving		Numerical
<code>fuel_consumption_comb</code>	Combined city and highway fuel consumption		Numerical
<code>fuel_consumption_mpg</code>	Fuel efficiency measured in miles per gallon		Numerical
<code>co2_emissions</code>	Carbon dioxide emissions measured in grams per kilometer (g/km)		Numerical

Table 1. Vehicle Dataset Features, Descriptions, and Data Types

2. Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) was performed to better understand the dataset, identify patterns, detect anomalies, and analyze the relationships between different features and CO2 emissions. Several statistical summaries and visualizations were used to explore the data before building the machine learning models.

EDA helps reveal important insights about the distribution of the data, correlations between variables, and the factors that most strongly influence vehicle emissions.

2.1. Distribution of CO2 Emissions

To better understand the target variable, several visualizations were created to analyze the distribution and spread of CO2 emissions in the dataset. The histogram and boxplot help identify the overall distribution pattern, central tendency, variability, and the presence of outliers. These visualizations provide important insights into how vehicle emissions vary across different vehicle types and specifications.

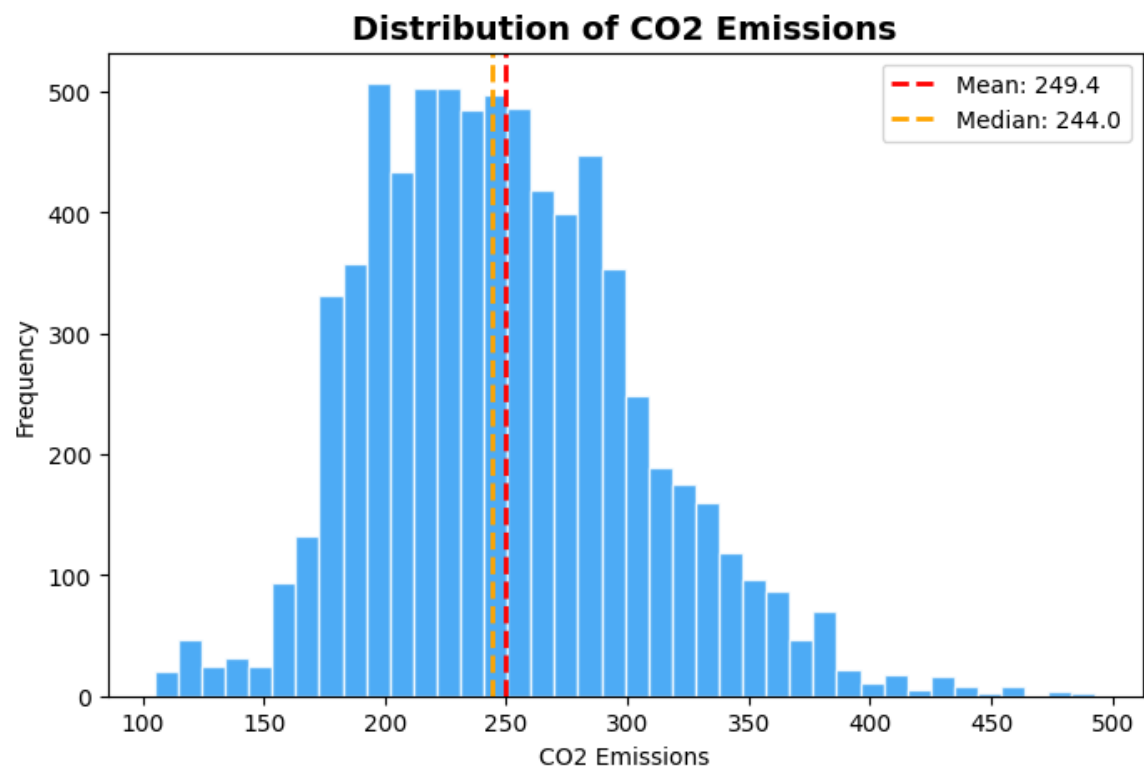


Figure 1. Distribution of CO2 Emissions Histogram showing the distribution of vehicle CO2 emissions in the dataset, including the mean and median values.

The histogram shows that most vehicles produce CO2 emissions between approximately 180 and 300 g/km. The distribution is centered around the mean value of 249.4 g/km and the median value of 244.0 g/km. Since the mean is slightly higher than the median, the data appears to be slightly right-skewed due to the presence of vehicles with higher emission values.

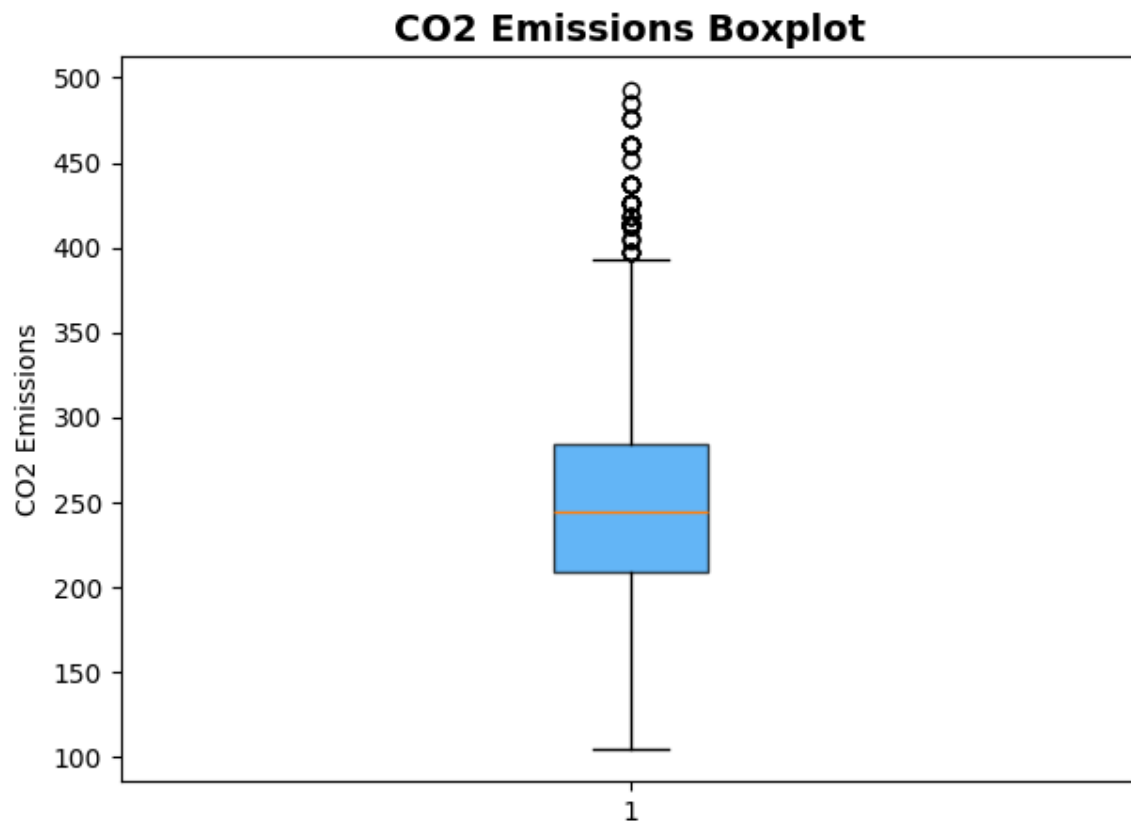


Figure 2. CO2 Emissions Boxplot
Boxplot illustrating the spread of CO2 emissions and highlighting the presence of outliers with unusually high emission values.

The boxplot provides additional information about the spread and variability of the data. Most vehicles fall within a moderate emission range, while several outliers appear above 400 g/km. These outliers represent vehicles with extremely high CO2 emissions, which are likely associated with larger engines or higher fuel consumption levels. The boxplot confirms that the dataset contains some extreme values that may influence machine learning model performance.

2.2. Relationship Between Vehicle Specifications and CO2 Emissions

To better understand the factors affecting vehicle emissions, scatter plots were created to analyze the relationship between engine size, number of cylinders, and CO2 emissions. Different fuel types were also visualized using color coding to compare their emission patterns.

2.2.1. engine size

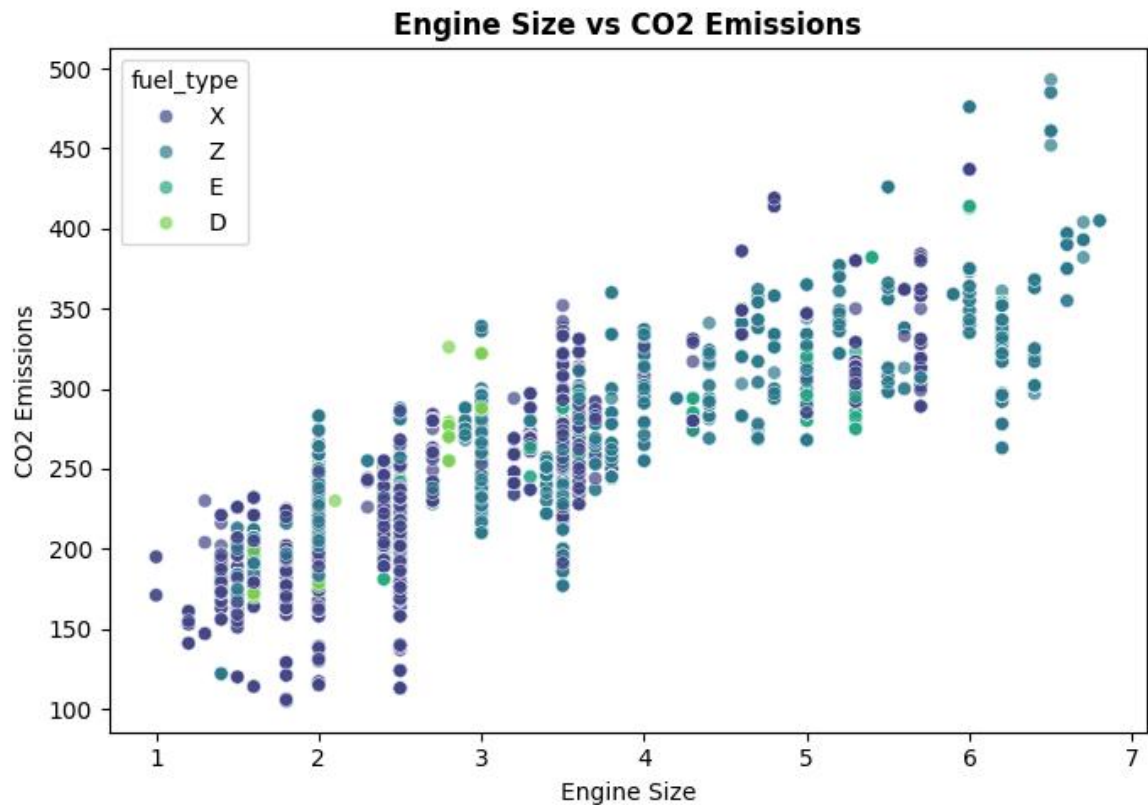


Figure 3. Engine Size vs CO2 Emission
s Scatter plot showing the relationship between engine size and CO2 emissions for different fuel types.

The scatter plot demonstrates a strong positive relationship between engine size and CO2 emissions. Vehicles with larger engines generally produce higher CO2 emissions compared to vehicles with smaller engines. Most vehicles with engine sizes between 1.0 and 3.0 liters show lower emission levels, while vehicles with engines larger than 5.0 liters tend to produce significantly higher emissions.

The visualization also shows that fuel type influences emission patterns. Some fuel categories appear more frequently in high-emission regions, indicating that both engine size and fuel type contribute to environmental impact.

2.2.2. cylinders

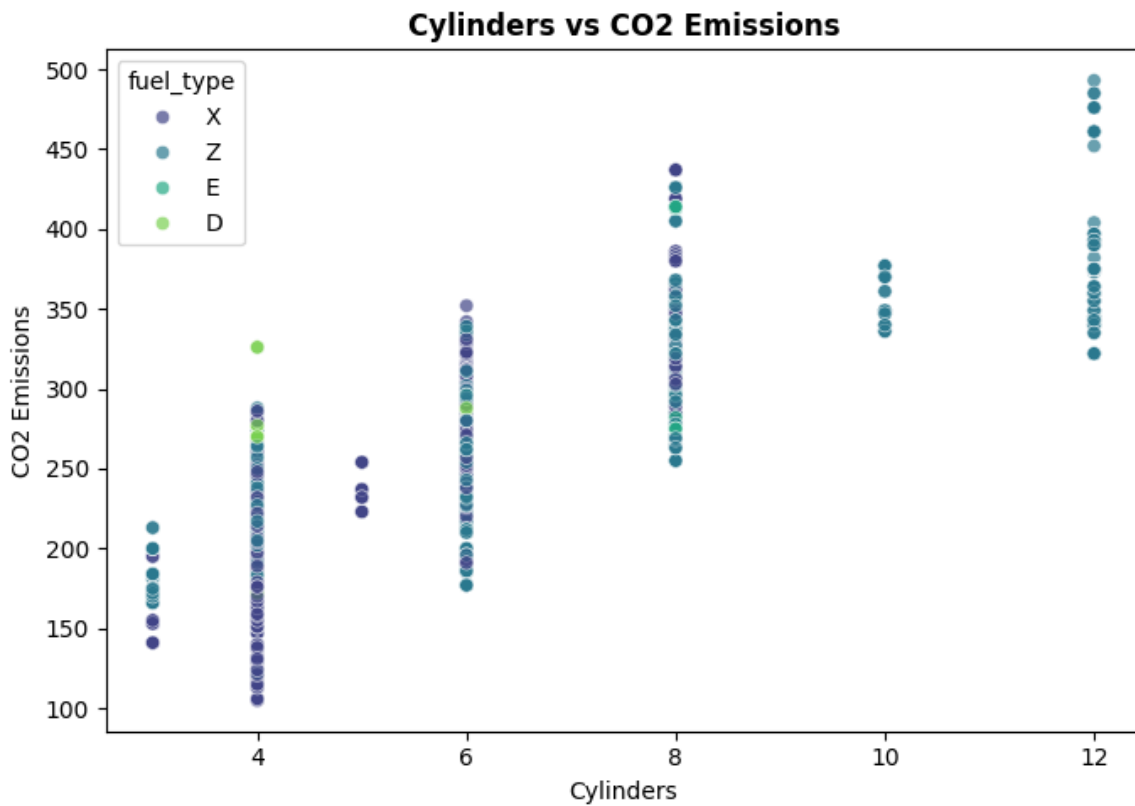


Figure 4. Cylinders vs CO2 Emissions
Scatter plot illustrating the relationship between the number of engine cylinders and CO2 emissions.

The scatter plot reveals that vehicles with a greater number of cylinders tend to produce higher CO2 emissions. Cars with 4 cylinders generally have lower emission values, while vehicles with 8, 10, or 12 cylinders show substantially higher emissions.

This trend suggests that engine complexity and power are strongly associated with fuel consumption and carbon dioxide production. The figure also indicates variability within the same cylinder category due to differences in fuel type.

2.3. Consumption Analysis

Fuel consumption is considered one of the most important factors affecting vehicle CO2 emissions. To better understand this relationship, scatter plots were created to compare city fuel consumption, highway fuel consumption, and combined fuel consumption with CO2 emissions for different fuel types.

2.3.1. City Fuel Consumption

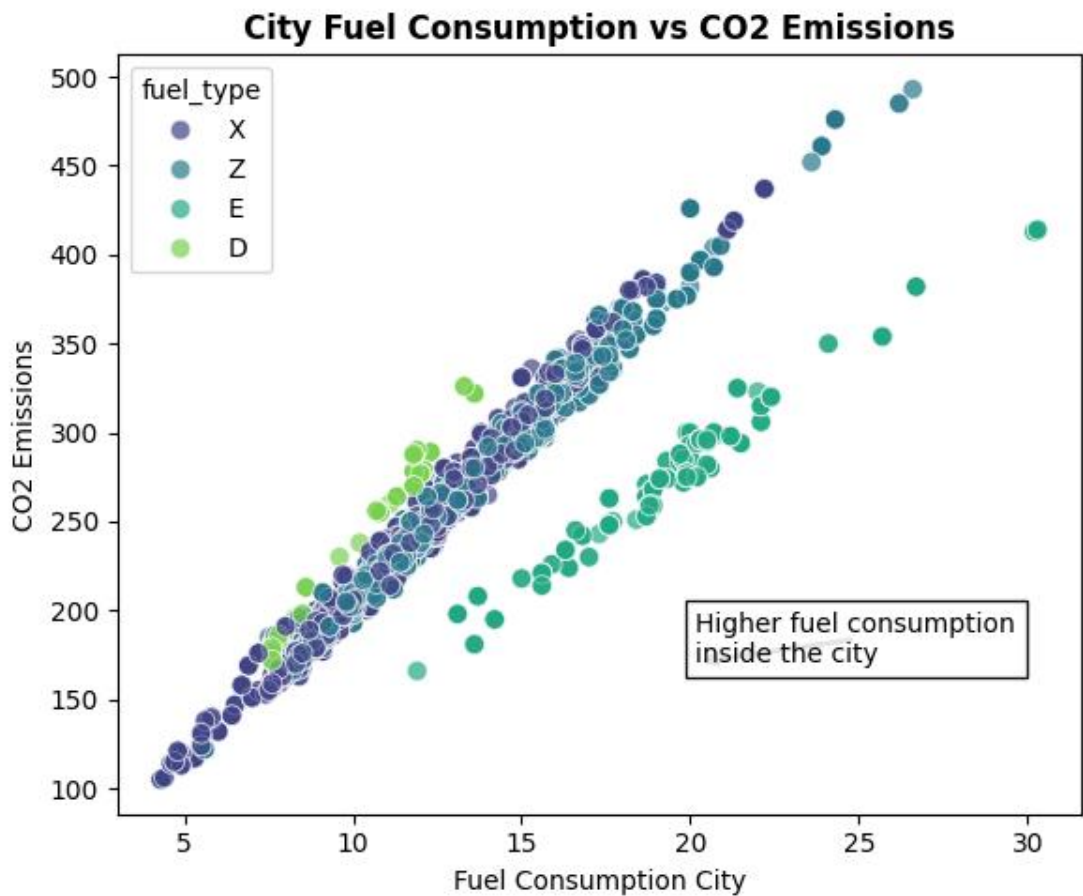


Figure 5. City Fuel Consumption vs CO2 Emissions
Scatter plot showing the relationship between city fuel consumption and CO2 emissions.

The figure shows a strong positive relationship between fuel consumption in city driving conditions and CO2 emissions. Vehicles that consume more fuel inside the city generally produce higher CO2 emissions. Most vehicles with low city fuel consumption have lower emission levels, while vehicles with high consumption values tend to generate significantly higher emissions.

The visualization also highlights differences between fuel types, where some fuel categories appear more frequently in high-emission regions.

2.3.2. Highway Fuel Consumption

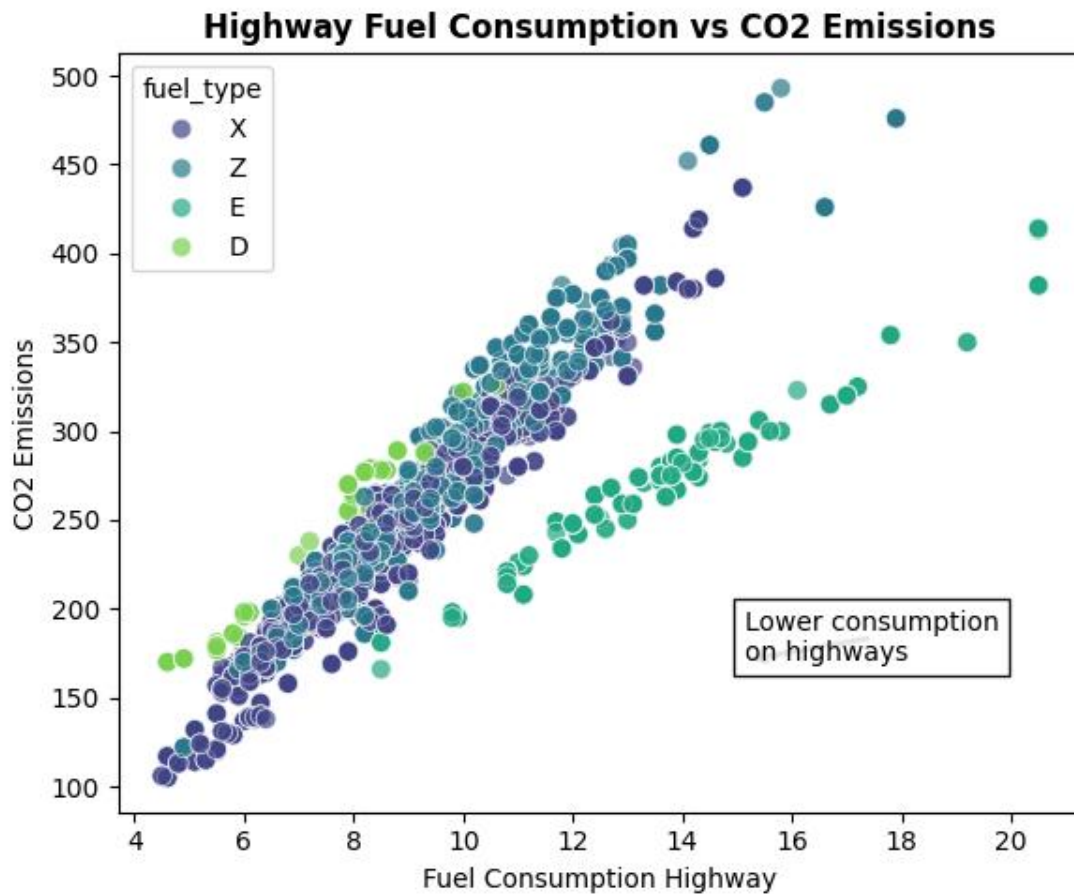


Figure 6. Highway Fuel Consumption vs CO2 Emissions
Scatter plot illustrating the relationship between highway fuel consumption and CO2 emissions.

The scatter plot demonstrates that highway fuel consumption is also strongly correlated with CO2 emissions. However, fuel consumption values on highways are generally lower compared to city driving due to more stable driving conditions and reduced stopping frequency.

Vehicles with higher highway fuel consumption still tend to produce larger amounts of CO2 emissions, confirming the direct relationship between fuel usage and environmental impact.

2.3.3. Combined Fuel Consumption

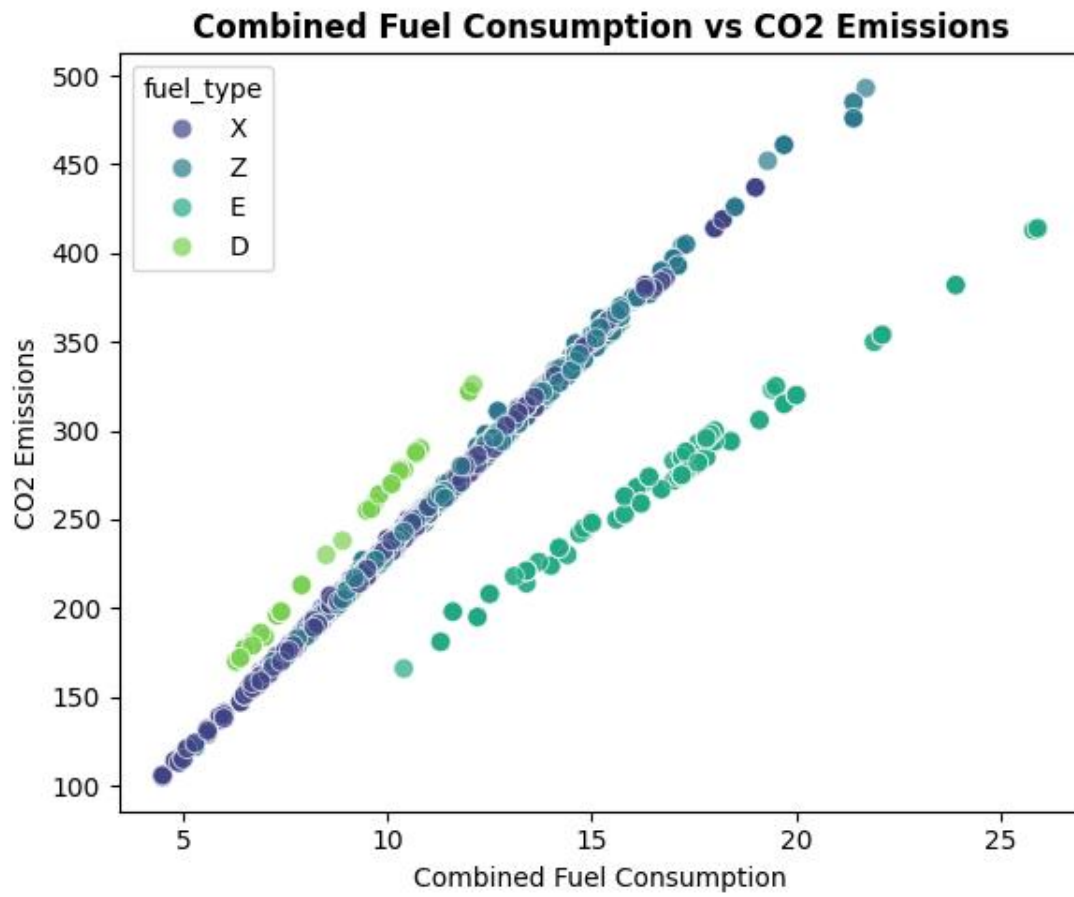


Figure 7. Combined Fuel Consumption vs CO2 Emissions
Scatter plot showing the relationship between combined fuel consumption and CO2 emissions.

The combined fuel consumption plot reveals one of the clearest linear relationships with CO2 emissions. As combined fuel consumption increases, CO2 emissions increase almost proportionally. This indicates that combined fuel consumption is one of the strongest predictors of vehicle emissions in the dataset.

The figure suggests that vehicles with better fuel efficiency generally produce lower carbon emissions, making fuel consumption an important factor in environmental sustainability and predictive modeling.

2.4. Fuel Type Analysis

To analyze the effect of fuel type on vehicle emissions, a bar chart was created showing the average CO2 emissions for each fuel category. The comparison helps identify which fuel types are associated with higher or lower environmental impact.

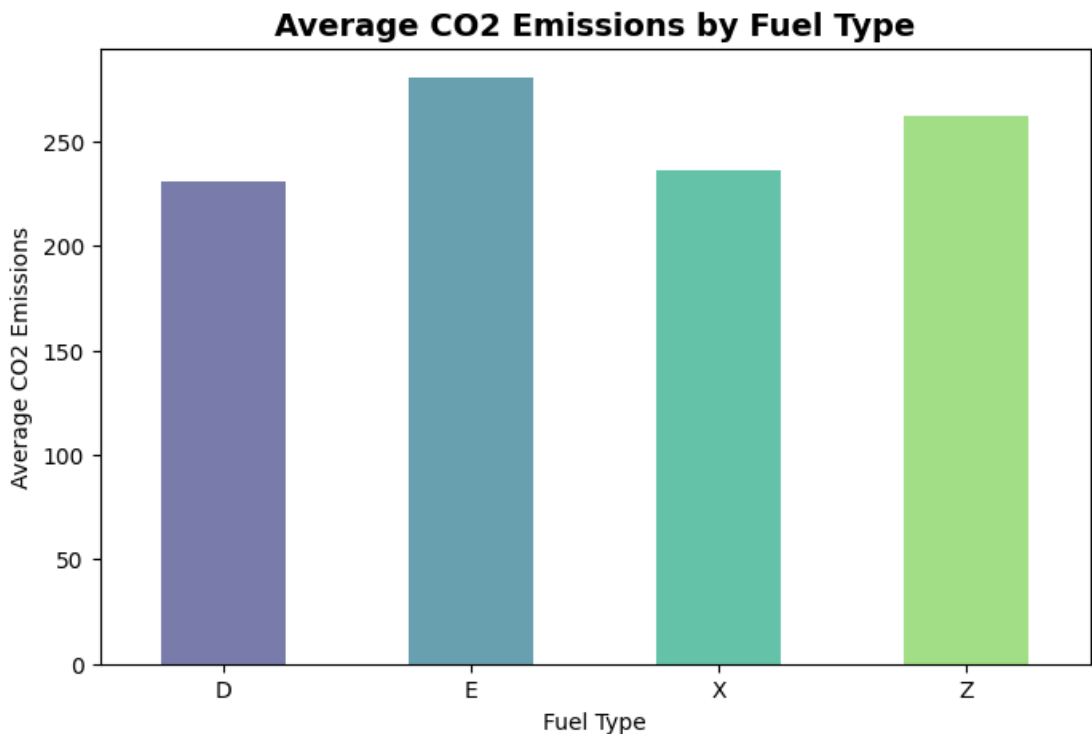


Figure 8. Average CO2 Emissions by Fuel Type
Bar chart comparing the average CO2 emissions for different fuel types.

The figure shows noticeable differences in average CO2 emissions among fuel types. Vehicles using Ethanol (E) produce the highest average CO2 emissions, followed by Premium Gasoline (Z). On the other hand, Diesel (D) and Regular Gasoline (X) show relatively lower average emission levels compared to the other categories.

These differences indicate that fuel type has a significant influence on vehicle emissions and overall fuel efficiency. The analysis also suggests that certain fuel categories may contribute more significantly to vehicle carbon emissions and environmental impact than others.

2.5. Correlation Analysis

A correlation heatmap was generated to analyze the relationships between the numerical features in the dataset. Correlation values range from -1 to 1, where values close to 1 indicate a strong positive relationship, values close to -1 indicate a strong negative relationship, and values near 0 indicate weak or no correlation.

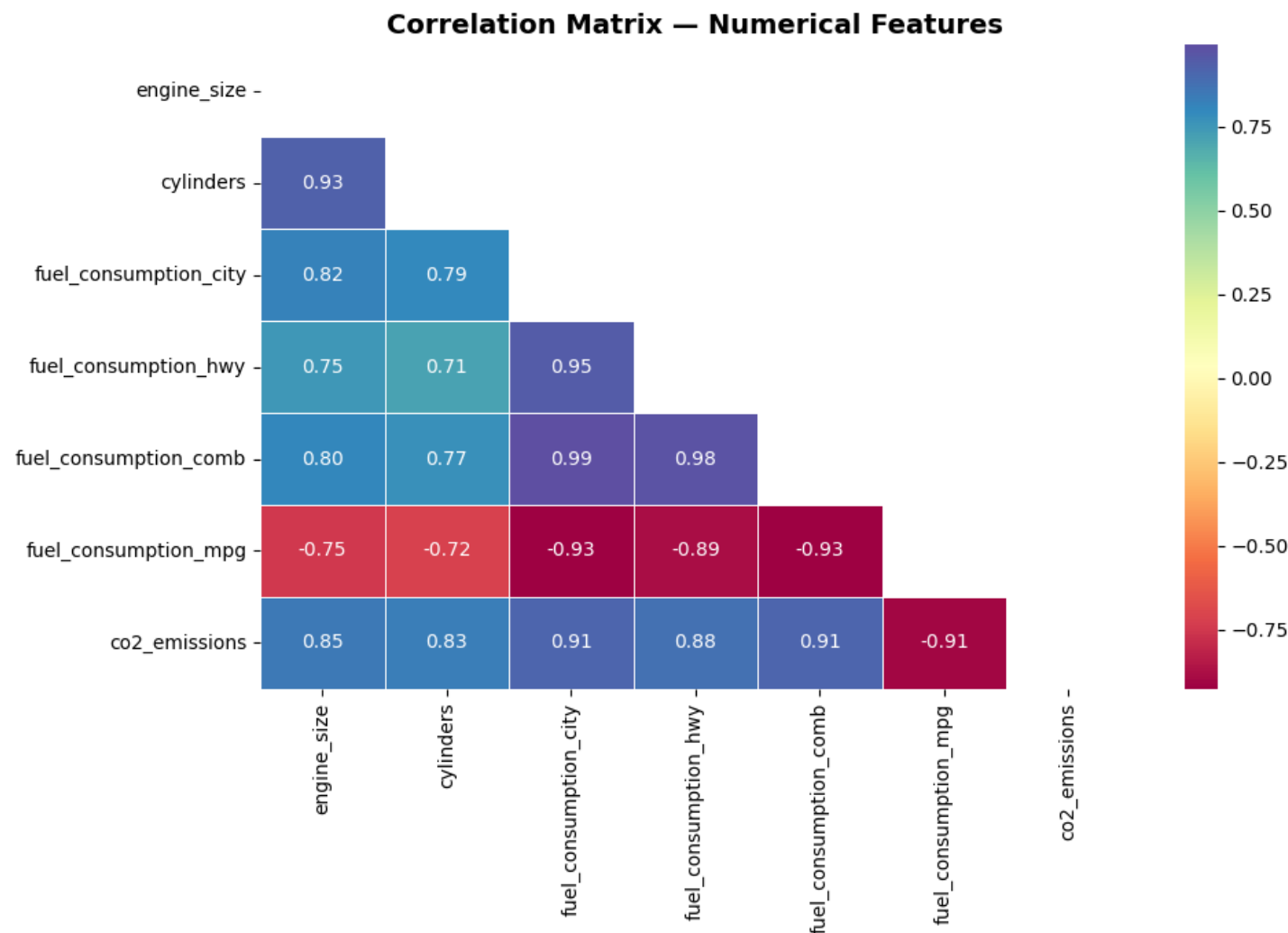


Figure 9. Correlation Matrix of Numerical Features
Heatmap showing the correlation coefficients between the numerical variables in the dataset.

The heatmap reveals several strong positive correlations between vehicle characteristics and CO2 emissions. Engine size and cylinders show strong positive correlations with co2_emissions, indicating that vehicles with larger engines and more cylinders generally produce higher carbon emissions.

Fuel consumption features also have very strong positive correlations with CO2 emissions. In particular, fuel_consumption_city and fuel_consumption_comb show correlation values above 0.90, making them some of the most important predictors of emissions in the dataset.

On the other hand, fuel_consumption_mpg has a strong negative correlation with CO2 emissions. This means that vehicles with better fuel efficiency measured in miles per gallon tend to produce lower carbon emissions.

The heatmap also reveals strong relationships among several fuel consumption variables, especially between city, highway, and combined fuel consumption measurements. These relationships indicate that fuel efficiency plays a major role in determining vehicle environmental impact.

Some fuel consumption variables showed strong correlations with each other, which may introduce multicollinearity in linear models. This issue can affect coefficient stability and interpretation, especially in regression-based algorithms.

2.6. Average CO2 Emissions by Vehicle Class

Vehicle class is another important factor that influences fuel efficiency and environmental impact. A bar chart was created to compare the average CO2 emissions across different vehicle classes in the dataset.

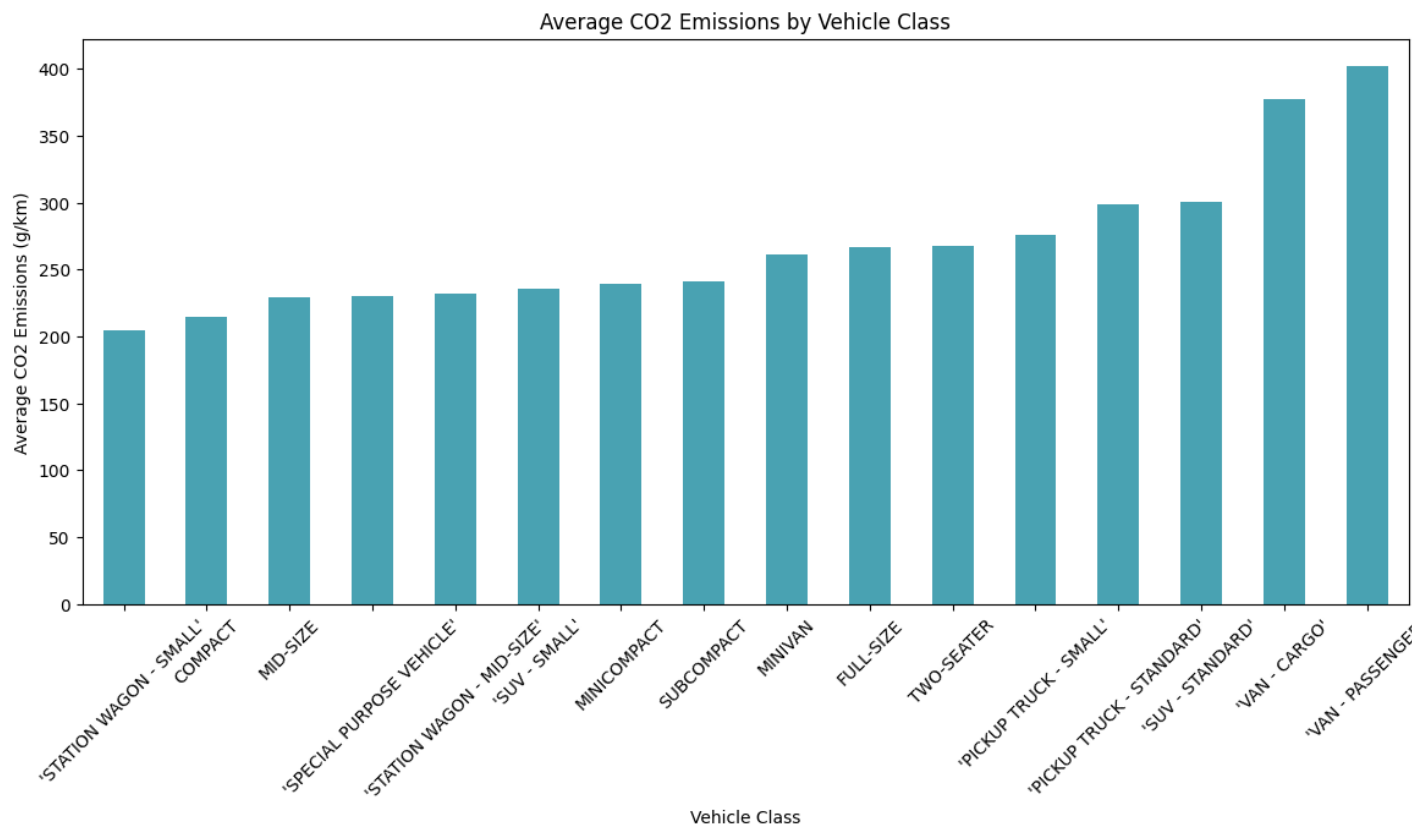


Figure 10. Average CO2 Emissions by Vehicle Class
Bar chart showing the average CO2 emissions for different vehicle classes.

The figure shows clear differences in emission levels between vehicle categories. Smaller vehicle classes such as station wagons, compact cars, and mid-size vehicles generally produce lower average CO2 emissions. In contrast, larger vehicle classes such as cargo vans, passenger vans, pickup trucks, and SUVs produce significantly higher emissions.

Passenger vans and cargo vans recorded the highest average CO2 emissions in the dataset, exceeding 350 g/km in some cases. This is likely due to their larger engine sizes, higher weight, and greater fuel consumption requirements.

The analysis indicates that vehicle size and purpose strongly affect environmental performance. Larger vehicles designed for cargo transport or heavy-duty usage tend to consume more fuel and generate higher carbon emissions compared to smaller passenger vehicles.

3. Data Preprocessing

Data preprocessing is an essential step in machine learning because raw data often contains missing values, duplicated records, inconsistent formats, and categorical variables that cannot be directly used by machine learning algorithms. Before training the models, several preprocessing techniques were applied to clean and prepare the dataset for analysis and prediction.

3.1. Handling Missing Values

Before training the machine learning models, the dataset was examined for missing values using the `isnull().sum()` function. Detecting missing values is an important preprocessing step because incomplete data can negatively affect model accuracy and reliability.

Step 1: Checking Missing Values

The analysis showed that several features contained missing values. Categorical features such as `make`, `model`, `vehicle_class`, `transmission`, and `fuel_type` had missing records. In addition, numerical features including `engine_size`, `cylinders`, `fuel_consumption_city`, `fuel_consumption_hwy`, `fuel_consumption_comb`, and `fuel_consumption_mpg` also contained missing values.

The target variable `co2_emissions` did not contain any missing values, so no imputation was required for the output variable.

Step 2: Numerical Feature Imputation

Missing values in numerical columns were handled using the median value of each feature. The **median** was selected because it is less sensitive to extreme values and outliers compared to the mean.

The following numerical features were imputed using the median:

- `engine_size`
- `cylinders`
- `fuel_consumption_city`
- `fuel_consumption_hwy`
- `fuel_consumption_comb`
- `fuel_consumption_mpg`

This approach helped preserve the dataset size while maintaining stable feature distributions.

Step 3: Categorical Feature Imputation

Missing values in categorical columns were replaced using the **mode** (most frequent value) of each feature. Using the mode ensures that missing records are filled with the most common category in the dataset.

The following categorical features were imputed using the mode:

- `make`
- `model`
- `vehicle_class`
- `transmission`
- `fuel_type`

This method allowed the dataset to remain consistent without removing incomplete rows.

Step 4: Verifying Missing Values

After completing the imputation process, the dataset was checked again using `df.isnull().sum()` to verify that no missing values remained. The results confirmed that all missing data had been successfully handled, and the dataset became ready for further preprocessing and machine learning model training.

3.2. Removing Duplicate Records

Duplicate rows were detected and removed using the `drop_duplicates()` function. Removing duplicated data improves dataset quality and prevents the machine learning models from becoming biased toward repeated observations.

3.3. Feature Encoding

Machine learning algorithms cannot process categorical text values directly, so categorical features must be converted into numerical format before model training. In this project, different encoding techniques were applied depending on the nature and characteristics of each feature.

Identifying Categorical Features

The categorical features in the dataset were divided into two groups based on their cardinality and data type.

High-Cardinality Features

The following features contain a very large number of unique values:

- make
- model

Encoding these features would create a large number of additional columns, increasing dataset dimensionality and model complexity. To avoid dimensionality explosion and reduce computational cost, these features were removed from the dataset before encoding.

Nominal Features

The following features are nominal categorical variables:

- vehicle_class
- fuel_type
- transmission

These features do not contain any natural order or ranking between categories. Therefore, One-Hot Encoding was selected as the most appropriate encoding technique.

Step 1: Dropping High-Cardinality Features

The make and model columns were removed using the `drop()` function. This step simplified the dataset and reduced the number of generated features during encoding.

After dropping these columns, the dataset shape was checked to verify the dimensionality reduction.

Step 2: Applying One-Hot Encoding

One-Hot Encoding was applied using the `pd.get_dummies()` function to convert nominal categorical variables into binary numerical columns.

The following features were encoded:

- `vehicle_class`
- `fuel_type`
- `transmission`

The parameter `drop_first=True` was used to remove one category from each encoded feature in order to avoid creating redundant columns. This helps reduce multicollinearity, where some generated columns become highly dependent on each other, and improves the stability and efficiency of the machine learning models.

After encoding, the dataset shape increased because each category was transformed into separate binary columns representing the presence or absence of a category.

Step 3: Verifying Encoded Data

A sample of the encoded dataset was displayed using `df_encoded.head()` to verify that categorical features were successfully transformed into numerical format suitable for machine learning algorithms.

3.4. Train-Test Split

Before training the machine learning models, the dataset was divided into training and testing sets using the `train_test_split()` function from Scikit-learn.

The input features were stored in `X`, while the target variable `co2_emissions` was stored in `y`.

The dataset was divided using the following configuration:

- Training set: 80%
- Testing set: 20%
- `random_state=42` for reproducibility

The training set was used to train the machine learning models, while the testing set was reserved for evaluating model performance on unseen data.

Splitting the dataset before scaling is important to avoid data leakage. If scaling is applied before splitting, information from the testing set may influence the preprocessing stage, leading to unrealistic evaluation results. Therefore, the dataset was first divided, and preprocessing steps such as scaling were later applied using only the training data.

Step 1: Defining Features and Target Variable

The target column `co2_emissions` was separated from the input features.

Step 2: Splitting the Dataset

The dataset was divided into training and testing sets using:

- `test_size=0.2` → 20% testing data
 - `random_state=42` → ensures consistent results across runs
-

Step 3: Verifying Dataset Shapes

The shapes of the training and testing datasets were displayed to confirm the split sizes.

In addition, summary statistics of the target variable were checked for both training and testing sets to ensure that the data distribution remained consistent after splitting.

3.5. Feature Scaling

Feature scaling was applied using **StandardScaler** to normalize the numerical features in the dataset. Scaling is an important preprocessing step because some machine learning algorithms, especially SVR and regularized models such as Ridge Regression, required feature scaling

Features with large numerical values may dominate smaller-valued features during model training, which can negatively affect prediction performance. Scaling helps ensure that all features contribute fairly to the learning process and improves numerical stability during optimization.

To avoid data leakage, the scaler was fitted only on the training data and then applied to both training and testing datasets using the same scaling parameters.

Step 1: Initializing the Scaler

A **StandardScaler** object was created from Scikit-learn.

Step 2: Fitting and Transforming the Data

The scaler was fitted on the training dataset using **`fit_transform()`**, while the testing dataset was transformed using **`transform()`** only.

This approach ensures that information from the testing data is not used during preprocessing.

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)
```

Step 3: Checking Data Before Scaling

The mean and standard deviation of the training data were examined before scaling to observe the original feature distributions.

Step 4: Checking Data After Scaling

After scaling, the transformed training data was converted into a DataFrame for easier inspection. The mean and standard deviation were checked again to verify that:

- The mean became approximately 0
- The standard deviation became approximately 1

This confirms that the numerical features were successfully standardized and prepared for machine learning model training.

4. Model Training

In this section, several machine learning regression models were trained and evaluated to predict vehicle CO2 emissions. The models were compared using multiple evaluation metrics to measure prediction accuracy and error levels.

The following evaluation metrics were used:

- **R² Score** → measures how well the model explains the variance in the target variable.
- **Mean Absolute Error (MAE)** → measures the average absolute difference between predicted and actual values.
- **Root Mean Squared Error (RMSE)** → measures the square root of prediction error magnitude.

RMSE penalizes large prediction errors more heavily than MAE because the errors are squared before averaging.

The main objective of this stage is to identify the model that achieves the highest prediction accuracy and the lowest error values.

4.1. Linear Regression

Linear Regression was used as a baseline machine learning model to predict CO2 emissions. The model assumes a linear relationship between the input features and the target variable.

The model was trained using the scaled training dataset (**X_train_scaled**) and the target variable (**y_train**). Training time was also measured to evaluate computational efficiency.

Step 1: Initializing the Model

A **LinearRegression()** object was created from Scikit-learn.

Step 2: Training the Model

The model was trained using the **fit()** function on the scaled training data.

The training time was measured using the time library to calculate how long the training process required.

Step 3: Making Predictions

After training, predictions were generated on the testing dataset using the **predict()** function.

The predicted values were stored in **y_pred_linear**.

Step 4: Evaluating Model Performance

The model performance was evaluated using:

- R² Score
- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)

These metrics help measure prediction accuracy and error magnitude.

Step 5: Displaying Results

The final results included:

- Training time
- R^2 Score
- MAE
- RMSE

These values were printed to analyze the effectiveness of the Linear Regression model in predicting vehicle CO2 emissions.

4.2. Ridge Regression

Ridge Regression is an improved version of Linear Regression that applies L2 regularization to reduce overfitting and improve model generalization. Regularization helps prevent the model from relying too heavily on specific features and improves performance on unseen data.

In Ridge Regression, a regularization parameter called alpha controls the strength of the penalty applied to the model coefficients. Smaller alpha values apply weaker regularization, while larger values apply stronger regularization.

To select the optimal alpha value, Cross-Validation was used to test multiple values and automatically choose the best one.

Step 1: Selecting the Best Alpha Value

A **RidgeCV** model was created using several alpha values:

- 0.01
- 0.1
- 1.0
- 10.0
- 100.0
- 1000.0

Five-fold cross-validation (cv=5) was applied to evaluate each alpha value and select the one with the best performance.

The training time for the cross-validation process was also measured.

After training, the optimal alpha value was stored in **best_alpha**.

Step 2: Training the Ridge Regression Model

After determining the best alpha value, the final Ridge Regression model was created using:
`ridge_model = Ridge(alpha=best_alpha)`

The model was then trained using the scaled training dataset (**X_train_scaled**) and target variable (**y_train**).

Training time was measured to evaluate computational efficiency.

Step 3: Making Predictions

After training, predictions were generated on the testing dataset using:

```
y_pred_ridge = ridge_model.predict(X_test_scaled)
```

The predicted values were stored in **y_pred_ridge**.

Step 4: Evaluating Model Performance

The Ridge Regression model was evaluated using:

- R^2 Score
- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)

These metrics were used to measure prediction accuracy and error magnitude.

Step 5: Displaying Results

The final output included:

- Best alpha value
- Training time
- R^2 Score
- MAE
- RMSE

These results were used to compare Ridge Regression with the other machine learning models and evaluate its ability to improve generalization and reduce prediction error.

4.3. Support Vector Machine (SVR)

Support Vector Regression (SVR) is a machine learning algorithm used for regression tasks. Unlike Linear Regression, SVR can model complex non-linear relationships between features and the target variable by using kernel functions.

In this project, the RBF kernel was used because the relationship between vehicle characteristics and CO₂ emissions is not perfectly linear, and the RBF kernel is effective for capturing complex non-linear patterns in the dataset.

The performance of SVR depends heavily on selecting appropriate hyperparameter values. Therefore, Grid Search with Cross-Validation was used to automatically test multiple parameter combinations and select the best configuration.

Step 1: Hyperparameter Tuning using Grid Search

Grid Search with Cross-Validation was applied to test different combinations of:

parameter	Description	values were tested
C	controls the penalty for prediction errors	1, 10, 100
Gamma	controls the influence of training examples.	0.01, 0.1, 1
epsilon	defines the acceptable error margin around the prediction line	0.1, 0.5, 1

Table 2. SVR Hyperparameters and Tested Values

A **GridSearchCV** object was created with:

- cv=5 for five-fold cross-validation
- scoring='r2' to evaluate performance using the R² metric
- n_jobs=-1 to use all available CPU cores for faster computation

The grid search automatically trained and evaluated multiple SVR models using different parameter combinations to identify the best-performing configuration.

Training time for the hyperparameter tuning process was also measured.

Step 2: Selecting the Best Parameters

After completing Grid Search, the best parameter combination was stored in best_params.

The selected parameters represent the configuration that achieved the highest cross-validation performance.

Step 3: Training the Final SVR Model

The final SVR model was created using:

- kernel='rbf'
- the optimal values of C
- gamma
- epsilon

The model was then trained using the scaled training dataset (**X_train_scaled**) and target variable (**y_train**).

Training time was measured to evaluate computational efficiency.

Step 4: Making Predictions

After training, predictions were generated on the testing dataset using:

```
y_pred_svr = svr_model.predict(X_test_scaled)
```

The predicted values were stored in **y_pred_svr**.

Step 5: Evaluating Model Performance

The SVR model was evaluated using:

- **R² Score**
- **Mean Absolute Error (MAE)**
- **Root Mean Squared Error (RMSE)**

These metrics were used to measure prediction accuracy and overall model performance.

Step 6: Displaying Results

The final results included:

- Best hyperparameter values
- Training time
- R² Score
- MAE
- RMSE

These results were compared with the other machine learning models to evaluate the effectiveness of Support Vector Regression in predicting vehicle CO2 emissions.

4.4. Decision Tree Regressor

Decision Tree Regressor is a non-linear machine learning model that predicts target values by splitting the dataset into smaller groups based on feature conditions. The model creates a tree-like structure where each split is selected to reduce prediction error.

Decision Trees can capture complex relationships between variables, but they are highly sensitive to overfitting. Therefore, hyperparameter tuning was applied to improve generalization and model performance.

Step 1: Hyperparameter Tuning using Grid Search

Grid Search with Cross-Validation was used to automatically test different combinations of Decision Tree hyperparameters.

The following parameters were tuned:

parameter	Description	values were tested
max_depth	controls the maximum depth of the tree.	5, 8, 10, 15,20
min_samples_split	defines the minimum number of samples required to split a node.	2, 5, 10, 20
min_samples_leaf	defines the minimum number of samples required in a leaf node.	1, 2, 4, 8

Table 3. Decision Tree Hyperparameters and Tested Values

A GridSearchCV object was created with:

- cv=5 for five-fold cross-validation
- scoring='r2' to evaluate model performance using the R² metric
- n_jobs=-1 to speed up computation using all CPU cores

The grid search process automatically evaluated multiple parameter combinations to identify the best-performing configuration.

Training time for the hyperparameter tuning process was also recorded.

Step 2: Selecting the Best Parameters

After Grid Search was completed, the optimal hyperparameter values were stored in **best_dt_params**.

These parameters were selected because they achieved the highest cross-validation performance while reducing overfitting.

Step 3: Training the Final Decision Tree Model

The final Decision Tree Regressor was created using the selected hyperparameters:

- `max_depth`
- `min_samples_split`
- `min_samples_leaf`

The Decision Tree model was trained using the original training dataset (**X_train, y_train**) instead of the scaled dataset because Decision Trees are not sensitive to feature scaling. The model makes predictions by creating rule-based splits on feature values rather than relying on distance calculations. Therefore, scaling does not significantly affect its performance.

Training time was measured to evaluate computational efficiency.

Step 4: Making Predictions

After training, predictions were generated on the testing dataset using:

```
y_pred_dt = dt_model.predict(X_test)
```

The predicted values were stored in **y_pred_dt**.

Step 5: Evaluating Model Performance

The Decision Tree model was evaluated using:

- R^2 Score
- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)

These metrics were used to measure prediction accuracy and error magnitude.

Step 6: Displaying Results

The final output included:

- Best hyperparameter values
- Training time
- R^2 Score

- MAE
- RMSE

These results were later compared with the other regression models to evaluate the effectiveness of the Decision Tree Regressor in predicting vehicle CO2 emissions.

4.5. Random Forest Regressor

Random Forest Regressor is an ensemble machine learning algorithm that combines multiple Decision Trees to improve prediction accuracy and reduce overfitting. Instead of relying on a single tree, Random Forest generates many trees and combines their predictions to produce more stable and reliable results.

Because Random Forest contains several configurable hyperparameters, Grid Search with Cross-Validation was used to automatically identify the best parameter combination.

Step 1: Hyperparameter Tuning using Grid Search

Grid Search with Cross-Validation was applied to test different combinations of:

parameter	Description	values were tested
n_estimators	controls the number of trees in the forest.	100, 200
max_depth	controls the maximum depth of each tree.	10, 15, 20
min_samples_split	defines the minimum number of samples required to split a node.	2, 5, 10
min_samples_leaf	defines the minimum number of samples required in a leaf node.	1, 2, 4

Table 4. Random Forest Hyperparameters and Tested Values

A GridSearchCV object was created using:

- cv=5 for five-fold cross-validation
- scoring='r2' to evaluate performance using the R² metric
- n_jobs=-1 to utilize all available CPU cores for faster computation

The grid search process automatically trained and evaluated multiple Random Forest models to determine the optimal hyperparameter values.

Training time for the hyperparameter tuning process was also recorded.

Step 2: Selecting the Best Parameters

After Grid Search was completed, the optimal hyperparameter values were stored in best_rf_params.

These parameters produced the best cross-validation performance and helped improve model generalization.

Step 3: Training the Final Random Forest Model

The final Random Forest Regressor was created using the selected hyperparameters:

- n_estimators
- max_depth
- min_samples_split
- min_samples_leaf

The Random Forest model was also trained using the original training dataset (**X_train, y_train**) because Random Forest is based on multiple Decision Trees. Since tree-based models are not affected by differences in feature magnitude, feature scaling is unnecessary for Random Forest. Using the original data preserves the natural feature distributions and simplifies the training process.

Training time was measured to evaluate computational efficiency.

Step 4: Making Predictions

After training, predictions were generated on the testing dataset using:

```
y_pred_rf = rf_model.predict(X_test)
```

The predicted values were stored in **y_pred_rf**.

Step 5: Evaluating Model Performance

The Random Forest model was evaluated using:

- **R² Score**
- **Mean Absolute Error (MAE)**
- **Root Mean Squared Error (RMSE)**

These metrics were used to measure prediction accuracy and prediction error.

Step 6: Displaying Results

The final output included:

- Best hyperparameter values
- Training time
- R² Score
- MAE
- RMSE

The results were later compared with the other regression models to evaluate the effectiveness of Random Forest in predicting vehicle CO2 emissions.

5. Evaluation

After training and testing all machine learning models, their performance was compared using four evaluation metrics:

- R^2 Score
- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- Training Time

The comparison results are shown in the table below.

Model	R^2 Score	MAE	RMSE	Training Time (s)
Linear Regression	0.9654	6.8699	10.8735	0.0436
Ridge Regression	0.9654	6.8708	10.8703	0.0032
SVR	0.9671	5.5172	10.6021	0.2702
Decision Tree	0.9440	4.3540	13.8407	0.0160
Random Forest	0.9753	3.2632	9.1859	0.8693

Table 5. Comparison of Regression Model Performance Metrics

5.1. Linear Regression Evaluation

Linear Regression achieved an R^2 Score of 0.9654, indicating that the model explains approximately 96.5% of the variance in CO₂ emissions. The model produced an MAE of 6.8699 and an RMSE of 10.8735.

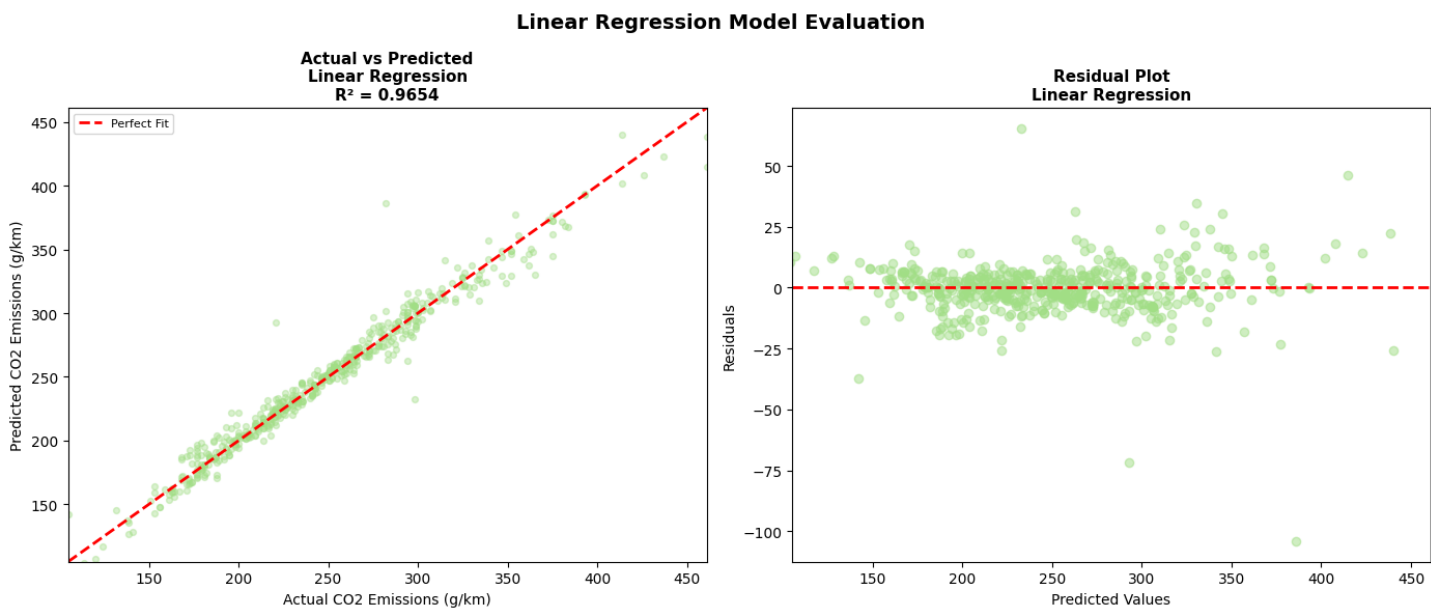


Figure 11. Actual vs Predicted and Residual Plots for the Linear Regression Model.

The figure above shows that most predicted values are relatively close to the actual CO₂ emission values, indicating good prediction performance for the Linear Regression model. In addition, the residual plot demonstrates that most prediction errors are distributed around zero, suggesting stable model behavior with moderate prediction errors.

The model trained efficiently with a training time of 0.0436 seconds, making it computationally inexpensive compared to more advanced machine learning models. However, its prediction accuracy remained lower than the non-linear and ensemble-based models.

5.2. Ridge Regression Evaluation

Ridge Regression produced results very similar to Linear Regression, achieving an R^2 Score of 0.9654, an MAE of 6.8708, and an RMSE of 10.8703.

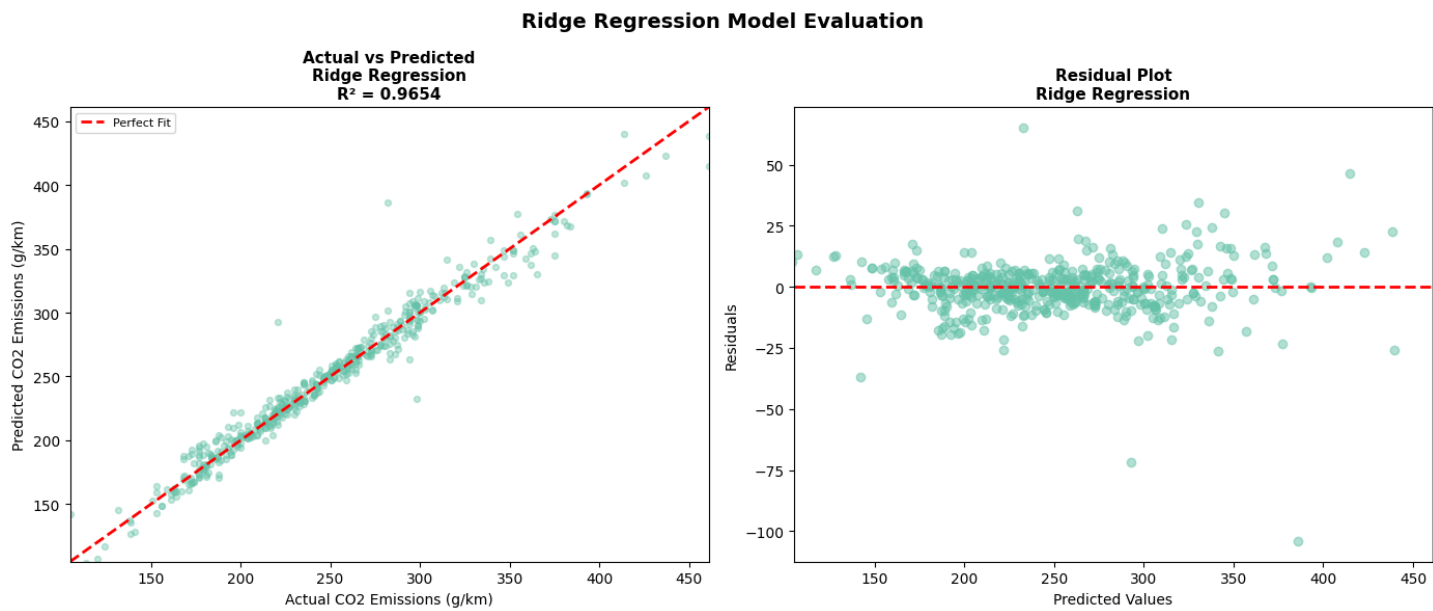


Figure 12. Actual vs Predicted and Residual Plots for the Ridge Regression Model.

The Actual vs Predicted plot demonstrates a strong alignment between the actual and predicted CO₂ emission values, indicating that the Ridge Regression model performs effectively on the dataset. In addition, the Residual plot shows that most prediction errors are relatively small and randomly distributed around zero, reflecting stable model performance and low prediction bias.

The optimal regularization parameter selected using cross-validation was:

- Best Alpha Value = 1.0

The hyperparameter tuning process required 0.1888 seconds, while the final model training required 0.0032 seconds.

Although Ridge Regression applies L2 regularization to reduce overfitting, the improvement over standard Linear Regression was minimal in this dataset, suggesting that the data already contains strong linear relationships between the features and CO₂ emissions.

5.3. Support Vector Regression (SVR) Evaluation

SVR achieved better prediction accuracy than both Linear Regression and Ridge Regression, with an R^2 Score of 0.9671. The model also reduced prediction errors, achieving an MAE of 5.5172 and an RMSE of 10.6021.

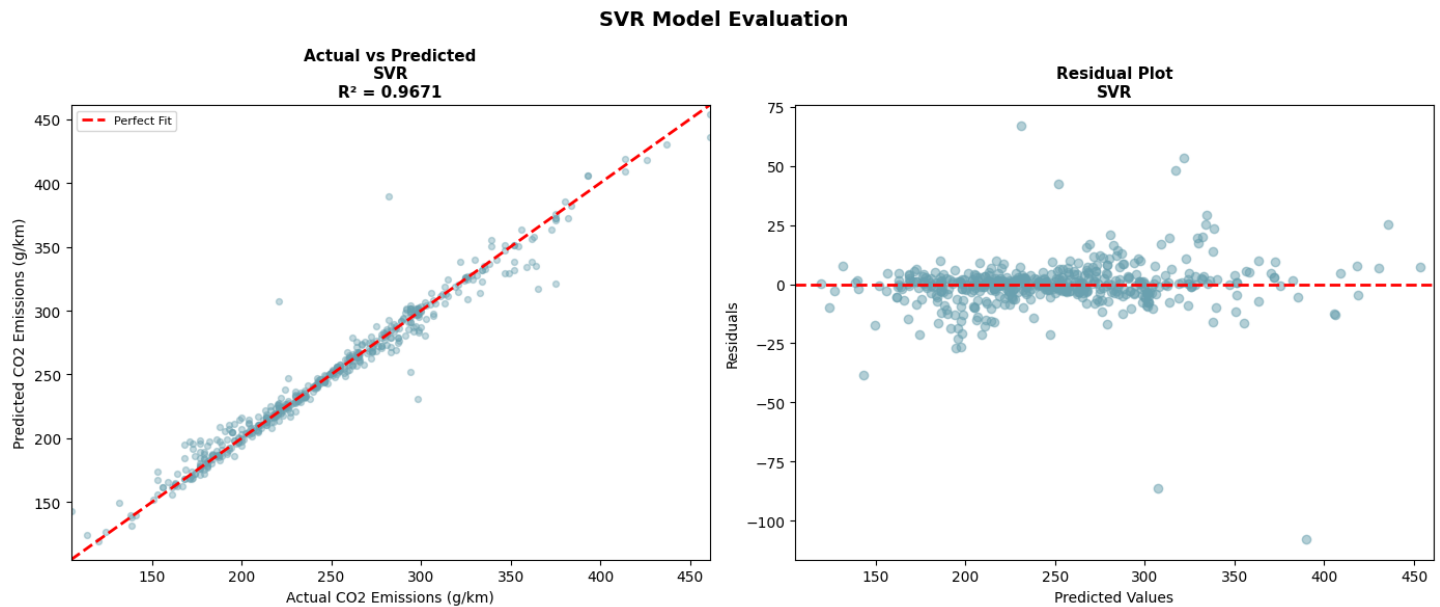


Figure 13. Actual vs Predicted and Residual Plots for the SVR Model.

The SVR model produced predictions that closely follow the actual CO2 emission values, especially in the middle range of the dataset. The residual plot also shows that most prediction errors are relatively small, indicating that the model successfully captured complex patterns in the data and achieved strong overall prediction performance.

The best hyperparameters selected using Grid Search were:

- $C = 100$
- $\epsilon = 1$
- $\gamma = 0.01$

The hyperparameter tuning process required 26.9913 seconds, while the final model training required 0.2702 seconds.

Although SVR improved prediction performance compared to the linear models, the hyperparameter tuning process remained computationally more expensive due to the complexity of kernel-based learning and Grid Search optimization.

5.4. Decision Tree Evaluation

Decision Tree achieved an R^2 Score of 0.9440, indicating good prediction performance and the ability to capture non-linear relationships in the dataset. The model produced an MAE of 4.3540 and an RMSE of 13.8407.

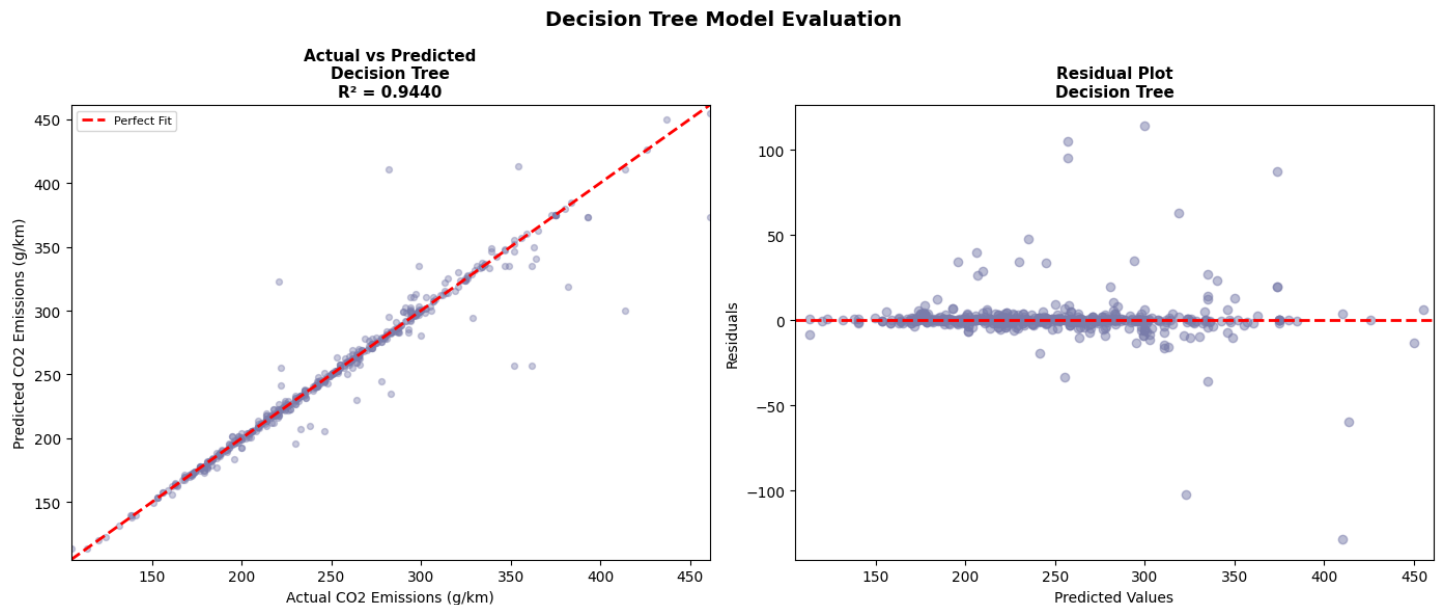


Figure 14. Actual vs Predicted and Residual Plots for the Decision Tree Model.

The Actual vs Predicted plot shows that many prediction points follow the general trend of the ideal fit line, indicating that the Decision Tree model captured important patterns in the data. In addition, the Residual plot demonstrates that most prediction errors remain near zero, although several larger residual values are present compared to the other models.

The best hyperparameters selected were:

- max_depth = 20
- min_samples_leaf = 2
- min_samples_split = 2

The Grid Search process required 1.5672 seconds, while the final model training required only 0.0160 seconds.

Although the Decision Tree model successfully captured complex non-linear relationships, its higher RMSE value suggests the presence of several larger prediction errors and weaker generalization performance compared to ensemble-based models such as Random Forest.

5.5. Random Forest Evaluation

Random Forest Regressor achieved the best overall performance among all tested models, with an R^2 Score of 0.9753, an MAE of 3.2632, and an RMSE of 9.1859.

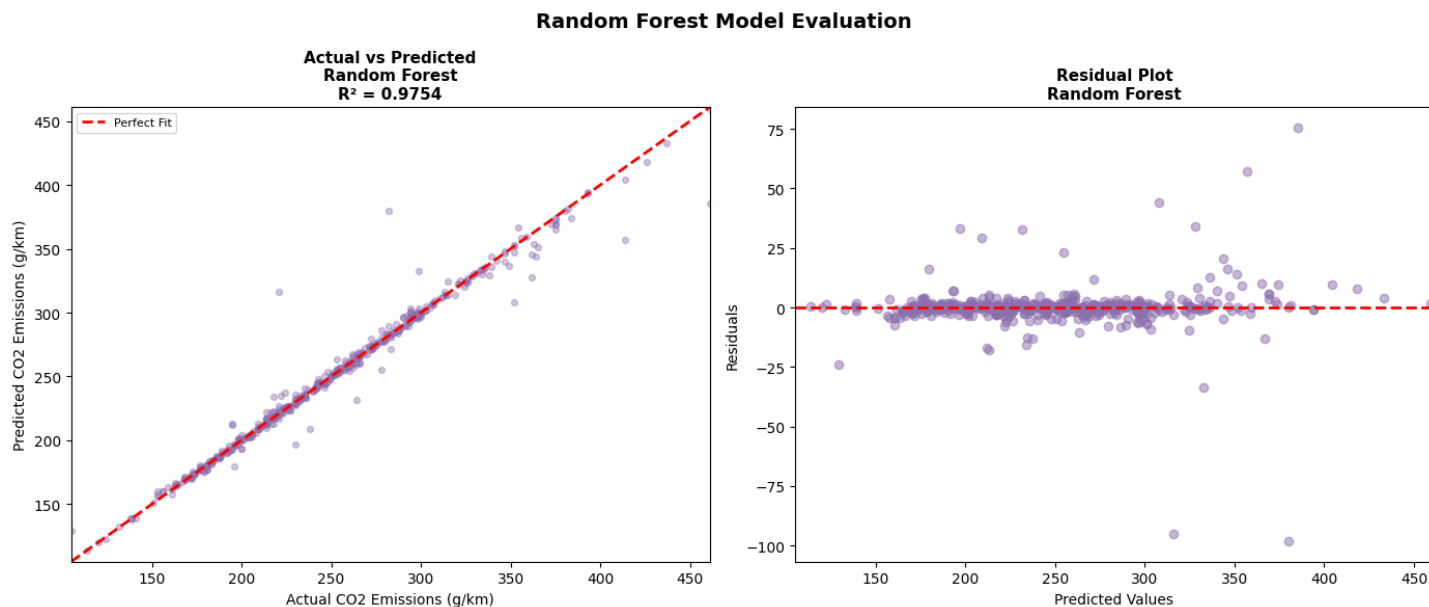


Figure 15. Actual vs Predicted and Residual Plots for the Random Forest Model.

The Actual vs Predicted plot shows that most prediction points are closely aligned with the ideal fit line, indicating strong prediction accuracy across the dataset. In addition, the Residual plot demonstrates that most residual values remain relatively close to zero with limited dispersion, reflecting stable model behavior and reliable prediction performance.

The best hyperparameters selected were:

- max_depth = 15
- min_samples_leaf = 1
- min_samples_split = 2
- n_estimators = 100

The Grid Search tuning process required 37.0934 seconds, while the final model training required 0.8693 seconds.

The Random Forest model demonstrated better generalization performance than the Decision Tree model by combining predictions from multiple decision trees. This ensemble approach helped reduce overfitting and produced more stable and accurate prediction results.

5.6. Overall Comparison

The evaluation results show that Random Forest achieved the best overall prediction performance among all tested models. It produced the highest R^2 Score and the lowest prediction errors, demonstrating strong ability to capture complex non-linear relationships while maintaining stable generalization performance.

The superior performance of Random Forest suggests that ensemble learning methods are more effective in handling complex relationships and reducing prediction variance compared to individual decision trees.

Decision Tree also achieved good predictive performance and successfully modeled non-linear relationships in the dataset. However, its higher RMSE value indicates the presence of several larger prediction errors, suggesting weaker generalization compared to Random Forest.

Linear Regression and Ridge Regression performed efficiently and trained very quickly, but their prediction accuracy remained lower because both models assume linear relationships between the input features and the target variable.

SVR achieved better prediction performance than the linear models and successfully captured more complex patterns in the data. However, the hyperparameter tuning process required higher computational time due to the complexity of kernel-based learning and Grid Search optimization.

Overall, the Random Forest Regressor was the most effective model for predicting vehicle CO₂ emissions in this project.

6. Feature Importance and Coefficient Analysis

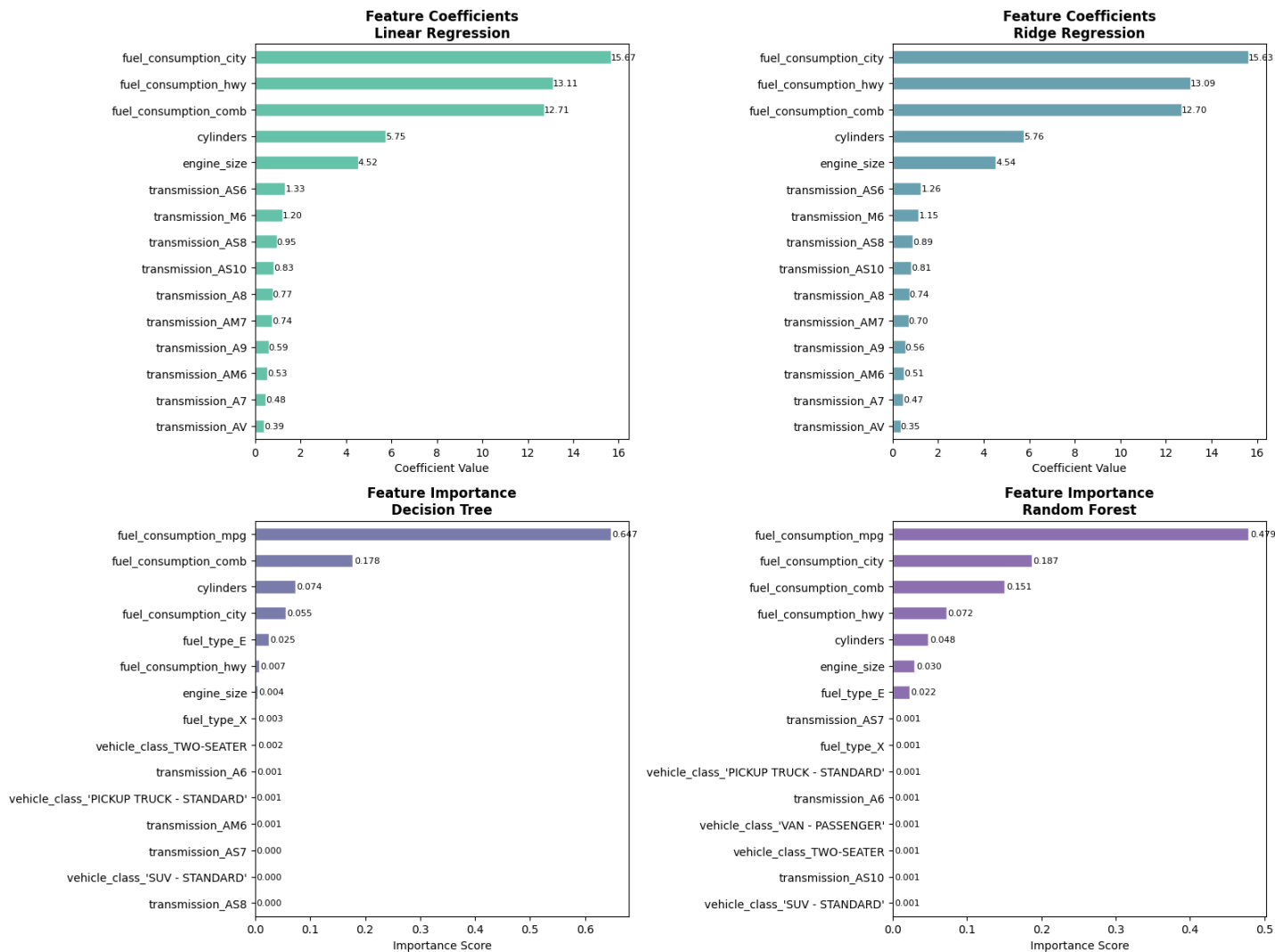


Figure 16. Feature Importance and Coefficient Analysis for Machine Learning Models Predicting CO2 Emissions.

The feature importance analysis was performed using multiple machine learning models, including Linear Regression, Ridge Regression, Decision Tree, and Random Forest, to identify the variables that most strongly influence vehicle CO₂ emissions.

For the linear models, fuel consumption–related features such as fuel_consumption_city, fuel_consumption_hwy, and fuel_consumption_comb received the highest coefficient values. This indicates that fuel consumption measurements are strongly associated with increased CO₂ emissions. Among these variables, fuel_consumption_city showed the largest coefficient values, suggesting that fuel consumption during city driving has a major impact on emission levels.

Engine size and the number of cylinders also contributed positively to the prediction process. Vehicles with larger engines and more cylinders generally produced higher CO₂ emissions due to increased fuel usage and engine power requirements.

In the tree-based models, Decision Tree and Random Forest identified fuel consumption variables as the most important predictors. In particular, fuel_consumption_mpg received the highest importance scores in both models, highlighting the

strong relationship between fuel efficiency and carbon emissions. Features such as fuel_consumption_city, fuel_consumption_comb, cylinders, and engine_size also contributed significantly to prediction performance.

The Random Forest model distributed feature importance across multiple variables more effectively than the Decision Tree model. This suggests that Random Forest captured broader relationships within the dataset and achieved stronger generalization performance while reducing overfitting.

It is important to note that linear models and tree-based models interpret feature importance differently. Linear models rely on coefficient magnitude and direction to measure feature influence, while tree-based models calculate importance according to each feature's contribution to reducing prediction error during the splitting process.

Overall, the analysis confirms that fuel consumption measurements are the dominant factors affecting vehicle CO₂ emissions, while engine specifications and transmission-related variables contribute to prediction performance to a lesser extent.

7. Development Considerations

Several important considerations were addressed during the development of this machine learning project to ensure accurate model performance and reliable prediction results.

7.1. Data Quality

The dataset contained missing values in both numerical and categorical features. Appropriate imputation techniques were applied to preserve the dataset and avoid information loss during preprocessing.

7.2. Categorical Data Processing

Categorical variables required special preprocessing before model training. High-cardinality features such as make and model were removed to reduce dimensionality, while nominal features were converted into numerical format using One-Hot Encoding.

7.3. Preventing Data Leakage

To ensure reliable evaluation results, the dataset was split into training and testing sets before applying feature scaling. The scaler was fitted only on the training data and then applied to the testing data using the same parameters.

7.4. Feature Scaling Requirements

Some machine learning algorithms, especially SVR and regularized models such as Ridge Regression, required feature scaling to improve numerical stability and prediction performance. In contrast, tree-based models such as Decision Tree and Random Forest were trained using the original unscaled data because they are not sensitive to feature magnitude.

7.5. Hyperparameter Optimization

Several non-linear and ensemble models required hyperparameter tuning to improve prediction accuracy and reduce overfitting. Grid Search with Cross-Validation was used to automatically test different parameter combinations and select the optimal configuration for each model.

7.6. Computational Performance

The computational cost varied significantly between models. Linear Regression and Ridge Regression trained very quickly, while SVR and Random Forest required substantially longer tuning and training times due to the large number of parameter combinations tested during Grid Search.

7.7. Model Generalization

Special attention was given to reducing overfitting and improving model generalization. Regularization techniques and hyperparameter tuning helped ensure that the models performed well not only on the training data but also on unseen testing data.

8. Conclusion

This project successfully developed and evaluated multiple machine learning models for predicting vehicle CO₂ emissions using vehicle specifications and fuel consumption data. Several preprocessing techniques were applied before model training, including handling missing values, removing duplicate records, encoding categorical variables, feature scaling, and train-test splitting to ensure reliable and consistent model performance.

Exploratory Data Analysis (EDA) revealed strong relationships between fuel consumption measurements, engine characteristics, and CO₂ emissions. Features such as `fuel_consumption_city`, `fuel_consumption_comb`, `fuel_consumption_mpg`, `engine_size`, and `cylinders` showed significant influence on emission levels across multiple machine learning models.

Several regression models were trained and compared, including Linear Regression, Ridge Regression, Support Vector Regression (SVR), Decision Tree Regressor, and Random Forest Regressor. The evaluation results demonstrated that ensemble and non-linear models achieved better prediction performance than the linear models.

Among all tested models, the Random Forest Regressor achieved the best overall performance, producing the highest R² Score and the lowest prediction errors while maintaining strong generalization capability. SVR also achieved strong predictive performance but required higher computational time due to hyperparameter tuning and kernel-based learning. Linear Regression and Ridge Regression provided fast training and stable baseline results but were less effective in modeling complex non-linear relationships within the dataset.

Feature importance analysis confirmed that fuel consumption-related variables were the dominant factors affecting vehicle CO₂ emissions, while engine specifications and transmission-related variables contributed to prediction performance to a lesser extent.

Overall, the project demonstrates the effectiveness of machine learning techniques in analyzing environmental data and predicting vehicle carbon emissions. The developed models can support environmental studies, fuel efficiency analysis, and sustainable transportation planning.

One limitation of this project is that the dataset contains highly correlated fuel consumption variables, which may affect the stability of some regression-based models. In addition, the dataset represents vehicles from a specific region and time period, which may limit model generalization to other markets or future vehicle technologies.

Future work may include testing larger datasets, advanced ensemble techniques, and deep learning models to further improve prediction accuracy and model robustness.